Ответы на контрольные вопросы
к практической работе № 6 (шаблоны Django)

Проект: «Платформа совместных покупок» (Joint Purchases). Студент: Евтушенко Егор Максимович, группа ЭФБО-08-24. Все пути указаны от корня репозитория проекта (рядом с manage.py). В проекте сущностям соответствуют классы Purchase (совместная покупка) и Participation (участие в покупке).

1. Наследование шаблонов

Базовый шаблон: templates/base.html — содержит DOCTYPE, head, навигацию и блоки. Дочерний шаблон начинается со строки:

```
{% extends "base.html" %}
```

Пример дочернего — purchases/templates/purchases/purchase_list.html. Тег {% extends %} говорит Django взять всю структуру base.html и подставить в его блоки содержимое дочернего шаблона. Заголовок вкладки отличается, т.к. в дочернем переопределён блок title:

```
base.html:  {% block title %}Совместные покупки{% endblock %}
purchase_list.html:  {% block title %}Покупки — Совместные покупки{% endblock %}
```

2. Переопределение блоков

В base.html объявлены блоки {% block title %} и {% block content %}. В дочернем purchase_list.html они переопределяются:

```
{% block content %}
    <h1>Совместные покупки</h1>
    ...
{% endblock %}
```

base.html задаёт каркас с пустыми блоками; при наследовании Django заменяет содержимое блока тем, что указано в дочернем. Поэтому если изменить текст внутри {% block content %}, именно он подставится в <main> страницы.

3. Передача контекста

Файл purchases/views.py, функция purchases():

```
context = {"purchases": load_purchases(PURCHASES_FILE)}
return render(request, "purchases/purchase_list.html", context)
```

В шаблоне purchase_list.html данные доступны по ключу контекста: {% for purchase in purchases %} и {{ purchases|length }}. Если изменить ключ, например на {"items": ...}, а в шаблоне оставить purchases, то переменная purchases станет неопределённой: цикл ничего не выведет, фильтр |length вернёт 0. Исключения не будет — обращение к отсутствующей переменной даёт пустое значение.

4. Обращение к атрибутам объектов

Файл purchases/templates/purchases/purchase_detail.html:

```
{{ purchase.product }}  {{ purchase.price }}  {{ purchase.id }}
```

Django обращается к атрибутам объекта Purchase. При опечатке {{ purchase.prodct }} Django по очереди пробует словарь, атрибут, метод, индекс и, не найдя ничего, подставляет пустую строку — исключение не возникает.

5. Обработка отсутствующих переменных

Если в purchase_detail.html вывести не переданную в контексте переменную, например {{ missing_var }}, на странице будет пустая строка.

По умолчанию Django заменяет неизвестные переменные пустой строкой (настройка string_if_invalid = ''), чтобы страница не падала из-за мелочи. Обратная сторона: опечатка в имени переменной молча ничего не выводит, и такую ошибку труднее заметить.

6. Автоматическое экранирование HTML

Значение выводится через {{ purchase.product }} в карточке purchase_card.html. Если в данных product равно Кофе, Django экранирует спецсимволы и на странице будет показан текст Кофе, а не жирный шрифт (< и > заменяются на < и >).

Отключать экранирование (фильтр |safe или {% autoescape off %}) опасно для значений, пришедших от пользователя: это открывает XSS — можно вставить <script> и выполнить чужой код.

7. Обработка коллекций

Файл purchase_list.html:

```
{% for purchase in purchases %}
    {% include "purchases/includes/purchase_card.html" %}
{% empty %}
    <p>Покупки не найдены.</p>
{% endfor %}
```

Если передать пустую коллекцию [], сработает ветка {% empty %} и выведется «Покупки не найдены.». Так отсутствие элементов обрабатывается прямо в шаблоне (аналогично в participation_list.html).

8. Условное отображение

Файл participations/templates/participations/includes/participation_status.html:

```
{% if participation.is_cancelled %}
    ...Отменено...
{% else %}
    ...Активно...
{% endif %}
```

Статус участия берётся из поля is_cancelled (data/participations.json). Если поставить is_cancelled = true, на странице участия и в списке вместо зелёного бейджа «Активно» появится серый «Отменено».

9. Использование фильтров

Файл purchase_list.html использует фильтр длины:

```
Всего покупок: {{ purchases|length }}
```

Фильтр изменяет выводимое значение перед показом: |length превращает список в его количество. Можно добавить ещё один уместный фильтр, например форматирование цены в карточке:

```
Цена: {{ purchase.price|floatformat:2 }} руб.
```

10. Обработка пустых значений (default)

Файл purchases/templates/purchases/includes/purchase_card.html:

```
{{ purchase.product|default:"Без названия" }}
```

Если product пустой или отсутствует — выведется «Без названия», иначе само значение. Фильтр default используют, чтобы на месте пустых или отсутствующих данных показывать осмысленный текст вместо пустоты.

11. Повторное использование шаблонов (include)

Включаемые фрагменты и где подключаются:

```
templates/includes/navigation.html  → base.html
purchases/.../includes/purchase_card.html → purchase_list.html
participations/.../includes/participation_status.html →
   список и деталь участия
```

Подключение: {% include "includes/navigation.html" %}. Изменение включаемого фрагмента отражается сразу на всех страницах, которые его подключают: например, правка ссылки в navigation.html меняет меню на всём сайте.

12. Передача данных во включаемый шаблон

Карточка purchase_card.html использует переменную purchase. Её имя задаёт внешний цикл в purchase_list.html: {% for purchase in purchases %}. Тег {% include %} наследует контекст, поэтому на каждой итерации карточка получает текущий объект purchase.

Если в карточке обратиться к несуществующему имени, например {{ room.product }}, или в цикле назвать переменную p, а карточка ждёт purchase — данные перестанут отображаться (пустая строка).

13. Формирование динамических URL

Файл purchase_card.html:

```
<a href="{% url 'purchases:detail' purchase.id %}">
```

Адрес строится из маршрута, а не задаётся жёстко. При purchase.id = 1 ссылка ведёт на /purchases/1/, при id = 2 — на /purchases/2/.

14. Работа с пространствами имён

Объявление в purchases/urls.py:

```
app_name = "purchases"
path("", views.purchases, name="list"),
path("<int:purchase_id>/", views.purchase_detail, name="detail")
```

Использование в шаблоне: {% url 'purchases:detail' purchase.id %} и {% url 'purchases:list' %}. Если изменить app_name или name, нужно согласованно обновить все теги {% url %}: в navigation.html, карточке, детальных шаблонах, 404.html и index.html.

15. Преобразование параметров маршрута

Файл purchases/urls.py — конвертер int:

```
path("<int:purchase_id>/", views.purchase_detail, name="detail")
```

Конвертер int принимает только целое число. Запрос /purchases/abc/ не совпадёт с маршрутом, Django не найдёт обработчик и вернёт 404 — в представление purchase_detail запрос не попадёт. Поэтому во view purchase_id всегда число.

16. Обработка отсутствующих объектов

Файл purchases/views.py, purchase_detail(); поиск — функция find_purchase_by_id из models/purchases.py (возвращает None, если объект не найден):

```
purchase = find_purchase_by_id(purchases_list, purchase_id)
return render(request, "purchases/purchase_detail.html",
    context, status=404 if purchase is None else 200)
```

В шаблоне purchase_detail.html: {% if purchase %}...{% else %}Покупка не найдена{% endif %}. Так страница показывает сообщение, а HTTP-ответ получает статус 404.

17. Различие ошибок маршрутизации и поиска объекта

Несуществующий маршрут (/xyz/): ни один path() в jointpurchases/urls.py и в urls.py приложений не совпал — Django вызывает handler404 (ошибка маршрутизации, ещё до представления). Существующий маршрут с несуществующим id (/purchases/9999/): маршрут совпал, purchase_detail вызвана, но объект не найден — представление само возвращает ответ со статусом 404 (ошибка поиска объекта, уже внутри view). Результат внешне похож (404), но причина возникает на разных этапах.

18. Обработка ошибок поиска шаблона

Представление обращается к шаблону в render(), например в purchases/views.py:

```
return render(request, "purchases/purchase_detail.html", ...)
```

Если указать неверное имя файла ("purchases/purchase_detales.html"), Django выбросит TemplateDoesNotExist — ошибка на этапе поиска файла. Это отличается от TemplateSyntaxError, который возникает, когда файл найден, но внутри него неверная конструкция.

19. Поиск ошибок в шаблонах

Файл purchase_list.html содержит цикл:

```
{% for purchase in purchases %}
    ...
{% empty %}
    ...
{% endfor %}
```

Если удалить {% endfor %}, Django выбросит TemplateSyntaxError: Unclosed tag 'for', и страница не сформируется. Исправление — вернуть закрывающий тег. Блочные теги ({% for %}, {% if %}, {% block %}) обязательно закрывать парным тегом.

20. Подключение статических ресурсов

Файл base.html — первая строка {% load static %}, далее:

```
<link rel="stylesheet"
      href="{% static 'homepage/css/style.css' %}">
<script src="{% static 'homepage/js/main.js' %}"></script>
<img src="{% static 'homepage/img/logo.png' %}">
```

Файлы лежат в homepage/static/homepage/. Если задать неверный путь (styles.css вместо style.css), ресурс не загрузится. Проверить можно командой python manage.py findstatic homepage/css/style.css или во вкладке Network браузера (там будет статус 404 на этот файл).

21. Изменение стилей страницы

Файл homepage/static/homepage/css/style.css:

```
.purchase-card {
    border: 1px solid #ddd;
    border-radius: 8px;
}
```

В purchase_card.html элемент имеет класс: <article class="card purchase-card mb-3">. Правило .purchase-card применяется ко всем элементам с этим классом — так HTML-класс связан с CSS-правилом. Например, замена border-radius на 20px скруглит углы у всех карточек.

22. Диагностика подключения JavaScript

Файл base.html подключает скрипт и содержит элемент, с которым он работает:

```
<span id="current-year"></span>
<script src="{% static 'homepage/js/main.js' %}"></script>
```

Скрипт homepage/static/homepage/js/main.js находит элемент current-year и выводит в него текущий год. Если год не появился: вкладка Console покажет ошибку в самом скрипте, а вкладка Network — статус 404 на main.js (файл вообще не загрузился, неверный путь). Если файл отдаётся (200), но год пуст — ошибка в логике скрипта.

23. Организация шаблонов приложений

Шаблоны разложены по приложениям с «двойным» путём, общие — на уровне проекта:

```
purchases/templates/purchases/purchase_list.html
participations/templates/participations/participation_list.html
homepage/templates/homepage/index.html
templates/base.html, templates/404.html,
templates/includes/navigation.html
```

В render() путь включает имя приложения: "purchases/purchase_list.html". Вложенная папка с именем приложения делает путь уникальным: без неё два шаблона list.html в разных приложениях конфликтовали бы и Django взял бы первый найденный.

24. Согласованность представления и шаблона

Файл participations/views.py, participation_detail():

```
context = {"participation": participation}
```

Шаблон participation_detail.html использует этот объект: {{ participation.user.name }}, {{ participation.purchase.product }}. Если во view переименовать ключ, например на {"item": participation}, а в шаблоне оставить participation — данные исчезнут (пустые значения). Исправление — согласовать имя ключа контекста и обращения в шаблоне.

25. Путь обработки HTTP-запроса (/purchases/1/)

1) jointpurchases/urls.py отрезает префикс purchases/:

```
path("purchases/", include("purchases.urls"))
```

2) purchases/urls.py: остаток 1/ совпадает с маршрутом, purchase_id = 1 (int) передаётся в представление:

```
path("<int:purchase_id>/", views.purchase_detail, name="detail")
```

3) purchases/views.py: purchase_detail(request, purchase_id=1) загружает покупки (storage.load_purchases), ищет объект (models.purchases.find_purchase_by_id), кладёт в context и вызывает render() с шаблоном purchases/purchase_detail.html.

4) Шаблон наследует base.html и выводит атрибуты покупки → HTML → HTTP-ответ 200. Если покупки с таким id нет (/purchases/9999/), find_purchase_by_id вернёт None, представление отдаст

статус 404, а шаблон покажет «Покупка не найдена».